

■■■ HEALTHCARE DATA DE-IDENTIFICATION ENGINE

System Specification & Technical Design Documentation

Document Class: Technical Whitepaper / Core Specifications

HIPAA Standard: Safe Harbor Method Alignment

Architecture: Local Hybrid NLP + Generative Multimodal AI

Future Scope: Demasking Agent, Decoupled Portals & Security Gateway

Status: Design Diagram Integrated

Target Workspace: major project

Generated On: 2026-06-11

1. Project Abstract

The **Healthcare Data De-Identification Engine** is an enterprise-grade, privacy-first software system designed to identify, redact, and manage Protected Health Information (PHI) from unstructured clinical documents. It ensures strict compliance with **HIPAA** and **GDPR** regulations while maintaining the clinical utility of the documentation for secondary research, statistical analyses, and downstream AI model training.

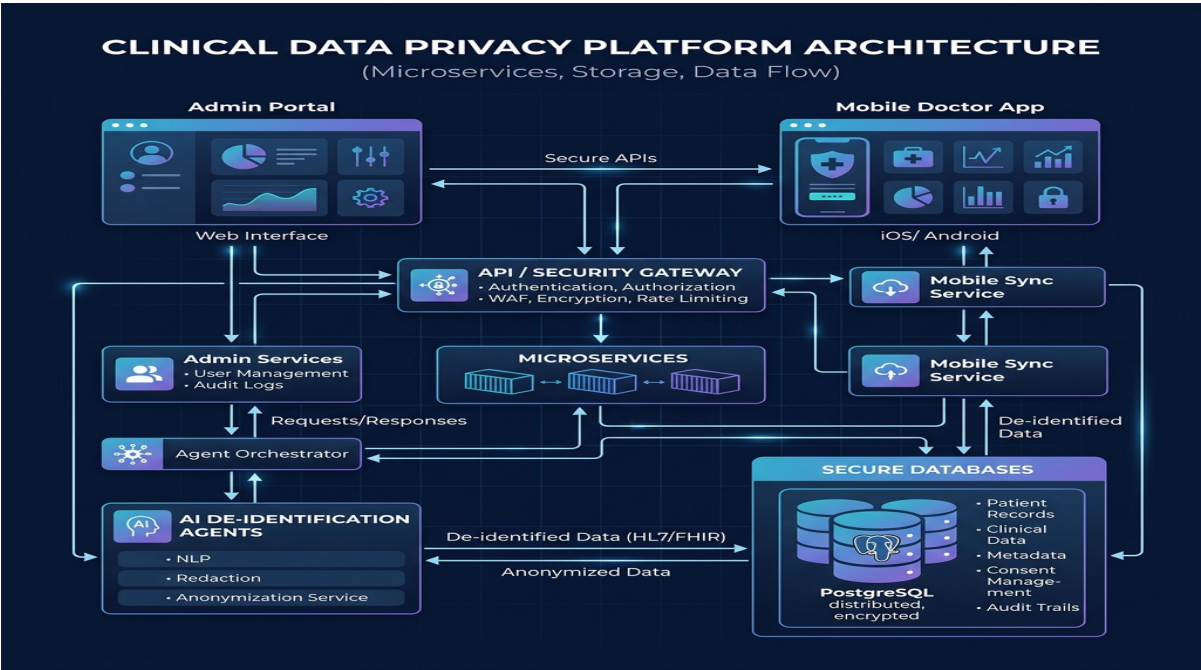
By employing a **hybrid detection paradigm** that combines local Natural Language Processing (NLP) with state-of-the-art Multimodal AI, the engine balances high-speed deterministic masking with contextual, visual-spatial reasoning.

2. Key System Features

- **Hybrid Detection Pipeline:** Uses a dual-stage pipeline containing local SpaCy NER (for fast, local text processing) and Gemini 2.5 Flash (via LangChain) for visual/scanned OCR and complex contextual reasoning.
- **Context Preservation:** Employs rule-based filters and whitelists to prevent the accidental redaction of critical medical metrics (e.g., dosages, blood pressure, lab values, disease terminology).
- **Reversible Redaction & Vault Mapping:** Uses a secure, isolated admin vault (phi_vault.json) that maps synthetic redaction placeholders back to original values, allowing authorized admins to fully restore/demask documents.
- **Multi-format PDF & Image Processing:** Supports raw text files (.txt), digital text PDFs, scanned image PDFs, and medical scans/images (.png, .jpg, .jpeg) with coordinate-level pixel blackouts.
- **Role-Based Access Control (RBAC):** Separates user flows into the **Admin Portal** (full control, uploads, unmasking) and the **Doctor/Third-Party Portal** (accesses redacted clinical summaries, adds diagnostic reviews, zero PHI exposure).
- **[PLANNED] Editable Demasking Agent:** A dedicated agent designed to reconstruct clinical documents into a fully editable format, allowing context-aware interactive restoration of patient details.
- **[PLANNED] Decoupled External Portal:** Transitioning the Doctor/Third-Party interface into a standalone external web/mobile application to completely isolate internal database services.
- **[PLANNED] Admin Authentication & Security Gateway:** Implementing a secure authentication mechanism and database-level encryption to safeguard patient identity records.

3. System Architecture & Component Interaction

The system is built on a clean, modular architecture, splitting concerns between UI representation, processing engines, and AI agents. Below is the updated visual architecture layout:



3.1 End-to-End Execution Workflow

Step 1: Upload & Digitization

The Admin logs in via the Security Gateway (Future). The Admin uploads a document (e.g., a scanned PDF report). The PDFHandler parses digital text. If the PDF is scanned, the OCRHandler enhances it, and DeidAgent runs a multimodal Gemini OCR model to extract raw text and coordinate-level bounding boxes.

Step 2: Identification & Filtering

The engine scans the text using regexes and local NLP models for high-confidence entities (Names, Emails, Dates). The ContextPreserver checks entities against a whitelist of medical conditions and measurement patterns to filter out false-positives.

Step 3: Redaction & Storage

The text is masked with synthetic placeholder tokens (e.g., [PERSON_REDACTED]). The mapping between placeholders and real names is sent to the PHIVault (phi_vault.json). The redacted text and redacted images are saved via DBHandler in shared_db.json.

Step 4: Medical Review (Decoupled Flow)

A Doctor accesses the system via a Separate Web/Mobile Application (Future). They review the redacted clinical findings (zero PHI exposure), appends notes/comments, and submits them, updating the status to 'Reviewed'.

Step 5: Demasking & Editing

The Admin reviews the doctor comments and triggers the Demasking Agent (Future) to reconstruct the clinical record. The agent reads the text and pulls the mappings from the vault, generating an editable document where name tokens can be changed or approved.

4. Technical Codebase Breakdown

File Path	Module Name	Core Responsibility
app.py	Streamlit Frontend & Dashboard UI	Manages portal authentication, page layouts, interactive tables, file upload queues, and triggers de-identification pipelines.
engine/deid.py	SpaCy Model & Regex Engine	Contains the core DeidEngine class. Handles SpaCy NER model loading, merges overlapping entity positions, and applies synthetic token masking.
engine/context.py	Medical Whitelist & Filters	Contains ContextPreserver. Filters out false-positives by evaluating text against lab value regexes and disease names whitelist.
engine/vault.py	JSON Key Mapping Vault	Contains PHIVault. Manages the secure JSON mappings and provides the dictionary-based replacement algorithm for reversible demasking.
agents/deid_agent.py	Gemini LangChain AI Agent	Contains DeidAgent. Directs Gemini 2.5 Flash to perform text extraction, entity sorting, and spatial boundary coordinates extraction.
agents/demask_agent.py [Planned]	LLM Demasking Agent	Will contain the DemaskAgent class. Designed to parse masked records and interactively reconstruct clinical narratives into an editable form.
utils/pdf_handler.py	PyMuPDF PDF Wrapper	Extracts digital vector text and applies visual white redaction boxes overlaying sensitive terms on PDF pages.
utils/ocr_handler.py	OpenCV Canvas Operations	Draws visual redaction masks onto image attachments, stacks cropped clinical disease regions, and generates synthetic files.
utils/db_handler.py	JSON Flat-File Database	Manages the read/write cycles of shared_db.json to persist document states, status logs, and doctor feedback.

5. Compliance & HIPAA Alignment

The engine maps directly to the standard HIPAA Safe Harbor requirements:

PHI Category	Handling Method	Pipeline Stage
Names & Initials	Replaced with synthetic tokens and logged in vault.	SpaCy PERSON entity + Gemini Agent extraction
Dates	Redacted except for relative age durations (e.g. '3 days ago').	Regex matcher + SpaCy DATE parser

Emails / Phones	Replaced and redacted completely.	Regex parser
Record & Patient IDs	Mapped and whited out.	Regex ID matcher
Medical Scans (Visual)	Redacted using visual bounding boxes.	OpenCV pixel erasure based on coordinates

6. Build & Setup Environment

```
# Install project dependencies
pip install -r requirements.txt

# Download required SpaCy corpus
python -m spacy download en_core_web_sm

# Start Streamlit application
streamlit run app.py
```